# Evidence, not verdicts: what 590 public PX4 crash logs can and cannot tell a drone fleet operator

Nagesh Chougule[a,*], Vibhav Ambardekar[a], Aditya Pathare[a], Atharv Waghmare[a], Shreyash Pawar[a], Piyush Bhamare[a]

[a]*Department of Mechanical Engineering, COEP Technological University, Pune, India*

**Abstract**

Flight logs are widely expected to determine whether a drone crash was caused by the pilot or the aircraft. Across 590 public PX4 crash logs from 479,547 flights, we evaluate that expectation using declared protocols committed prior to analysis. First, automatic crash attribution between pilot and aircraft is not supported by these telemetry rules, models and labels: a fixed rule reached a balanced accuracy of 0.47 on held-out crashes, cross-validated classification yielded AUCs of 0.41–0.50 (95% bootstrap upper bounds $\leq 0.62$, below the 0.66 permutation null detection threshold), and evaluating the rule against 43 downloadable expert forum logs showed poor agreement with community diagnoses (balanced accuracy 0.381, 95% CI 0.20–0.59). Second, diagnostic checks suffer from crash data leakage: tuning fault sensitivity fell from 77.8% to 11.1% ($p = 0.031$) without the final 10 s, and cross-sectional separation was substantially attenuated within the same drone (AUC 0.58). Third, drones that later crashed showed higher baseline vibration than matched peers on older estimator metrics (mean rank 0.67, Holm $p = 0.016$), though replication on newer accelerometer metrics was inconclusive ($p = 0.25$). Stratum-matched fleet screening yields positive predictive values of 2.26–2.47% (lift 3.3×, 40–44 inspections per crash) at a 10% flag rate. AeroFleet 2.0, an open-source assurance tool, presents longitudinal evidence without issuing automatic verdicts.

*Keywords:* Unmanned aircraft systems, Flight log analysis, Crash investigation, Fault diagnosis, Data leakage, Declared protocols, Fleet assurance

*Corresponding author.

*Email address:* nkc.mech@coeptech.ac.in (Nagesh Chougule)

## 1. Introduction

Small unmanned aircraft are now flown commercially for survey, inspection, agriculture and delivery, and the rules that govern them place weight on records. In the European Union and the United Kingdom an operator in the specific category must keep records of operations, maintenance and personnel for at least three years [1, 2]; India requires an accident to be reported within 48 hours [3], and the United States within ten days [4]. When something goes wrong, the onboard flight log is the richest record available. Open-source log viewers, commercial fleet platforms and, more recently, machine-learning and language-model prototypes promise to turn that log into an answer: what failed, and whether the pilot or the aircraft was responsible [5, 6].

The question matters beyond engineering curiosity. In aviation safety science, accidents are recognized as systemic, multi-causal events involving human factors, environmental conditions and organizational influences [7]. Commercial warranty claims, insurance settlements and regulatory filings nevertheless frequently compress these dynamics into a binary decision: whether the pilot or the aircraft was primarily responsible. An automatic verdict that looks objective can carry weight it has not earned. Fleet operators face a related but different question: not why one flight ended badly, but which of their drones are most likely to fail next and should be inspected first. A log-based method that is good at the second task need not be good at the first, and confusing the two can mislead both investigators and maintainers.

Whether logs can deliver an answer to the first question has, to our knowledge, not been tested on real crashes at scale. Public fault datasets are dominated by simulation or by faults injected on purpose [8, 9, 10], and accident statistics classify outcomes such as loss of control rather than causes visible in a log [11, 12]. Two pitfalls make optimistic results easy to obtain. First, a log that contains a crash also contains its consequences: saturated motors, extreme attitudes and estimator errors. A detector can learn these instead of the cause, a form of data leakage [13, 14]. Second, comparing crash flights with other operators' good flights mixes the flight with the aircraft and the people using it, so a difference between groups need not say anything about the flight that crashed.

This paper asks four questions of the largest public archive of real autopilot logs, PX4 Flight Review [15, 16]:

Q1 Do configuration settings differ between crash flights and normal flights?

Q2 Can a log separate crashes caused by the pilot from crashes caused by the aircraft?

Q3 Does the flight that crashed show warning signs relative to the same drone's history, or are the differences between drones?

Q4 Do expert diagnoses agree with uploader labels, and do automatic checks find the faults experts name?

Because language-model assistants are now proposed for exactly this task, we also report a controlled test of small local models used as a review panel.

The contributions are as follows. (i) We assemble a real-flight crash benchmark from public logs and check its labels against expert diagnoses. (ii) Every confirmatory test was declared, with its threshold and pass mark, in public version control before the data it used were analysed, and decisive tests were repeated on held-out data, following declared test protocols committed prior to analysis [17]; secondary sensitivity and diagnostic analyses added during revision are explicitly designated as exploratory. (iii) We show that with these rules, models and labels, automatic crash attribution from logs was not achievable above chance: cross-validated classification yielded upper 95% confidence bounds between 0.510 and 0.619, well below the 0.66 permutation null detection threshold, while agreement with expert forum diagnoses did not exceed chance (balanced accuracy 0.381, 95% CI 0.20–0.59; pilot recall 1 of 5). Furthermore, apparent diagnostic accuracy can come from the crash itself. (iv) We report one positive screening signal: drones that later crash are rougher on their ordinary flights than matched drones on older estimator metrics, though replication on newer accelerometer metrics was inconclusive ($p = 0.25$). Screening against stratum-matched fleet controls yields an operational positive predictive value of 1.37–1.48% (relative risk 3.00×, lift 2.00×) at a 25% flag rate and 2.26–2.47% (relative risk 4.50×, lift 3.33×, 40.5–44.2 inspections per crash) at a 10% flag rate across screenable (0.69%) and fleet-wide (0.74%) vehicle base rates (1.04% under empirical matched-set screening). (v) We show how these results translate into the design of an open-source assurance tool, AeroFleet 2.0, and report engineering checks of its parts, including one negative result (Section 6). Code, declarations, derived features and labels are released.

## 2. Related work

*Fault datasets.* Table 1 compares our data with the main public resources. ALFA provides 47 fixed-wing flights with injected engine and control-surface

Table 1: Public UAV fault and anomaly data compared with this study.

| Resource | Flights | Faults or labels | Real crashes labelled by cause |
|---|---|---|---|
| ALFA [8] | 47 fixed-wing flights | Injected engine and control-surface failures | No |
| RflyMAD [9] | 5,629 cases (497 real) | 11 injected fault types | No |
| UAV-SEAD [10] | about 1,396 real PX4 logs | State-estimation anomalies (4 classes) | No |
| This study | 479,547-log index; 590 crash-rated logs; 148 expert-diagnosed threads | Uploader responsibility labels; expert causes | Yes (uploader; partly expert-checked) |

failures [8]. RflyMAD covers 5,629 multicopter fault cases with eleven fault types, of which 497 are real flights and the rest software- or hardware-in-the-loop simulations [9]. UAV-SEAD offers about 1,396 real PX4 logs annotated for state-estimation anomalies [10], used by later anomaly-mining work [18]; a recent testbed quantifies the discrepancies between simulated and real PX4 logs [19]. These resources target anomaly or fault detection under known faults. None labels real crashes by responsibility, and none separates the pre-crash window from the crash.

*Fault detection and diagnosis.* Surveys of fault detection for UAVs and robots cover model-based and data-driven methods and stress how hard robotic diagnosis is in the field [20, 21]. Causal models built from autopilot documentation have been used to find actual causes of failures in ArduPilot logs [22]. A study of 569 bugs in PX4 and ArduPilot found parameter handling among the root-cause types of drone-specific bugs [23]. Maintenance-oriented reliability analysis of UAVs models the passage from partial degradation to failure [24], the view our between-drone result supports.

*Accident analysis.* Analyses of civil drone occurrences find equipment problems and loss of control prominent [11]. Human-factors frameworks for remotely piloted aircraft (such as the Human Factors Analysis and Classifi-

cation System, HFACS) model mishaps as multi-causal sequences involving organizational influences, environmental preconditions, and supervisory failures alongside direct operator actions [7, 25]. In operational practice, post-incident reporting and warranty claims often collapse these complex systemic interactions into a single nominal party: pilot versus aircraft. Because such classifications depend heavily on the rater and framing, we evaluate agreement against expert community diagnoses.

*Language models for diagnosis.* Language-model agents have been applied to simulation testing and log analytics for small UAVs [5], and learned models to classifying problems in drone log messages for forensics [6]. Multi-agent debate has been proposed to improve the factuality of model outputs [26], while indirect prompt injection shows that text inside data can steer model behaviour [27]. We test both properties on our task.

*Methodology.* Leakage, where information unavailable at prediction time enters training or evaluation, is a known source of irreproducible machine-learning results [13, 14]; label noise likewise biases evaluation [28]. Matching reduces confounding in observational comparisons [29], and declaring hypotheses and analysis scripts in version control prior to running tests separates confirmatory from exploratory analysis [17]. We apply these ideas to drone logs.

*Tools for operators.* Retrieving similar past cases and reusing their solutions is the basis of case-based reasoning [30]; ranking text by word overlap with BM25 is a standard, deterministic retrieval method [31]. A hand-held miniature of the environment lets a person grasp and inspect a spatial scene at a glance [32], the idea behind the tabletop view in Section 6. We use these as engineering choices and test them only as far as stated.

## 3. Data

### *3.1. Archive and labels*

PX4 Flight Review is a public service to which PX4 users upload onboard logs (ULog format) for plotting and sharing [16]. We downloaded its public index on 20 October 2026: 479,547 logs (32,793,172 bytes, SHA-256 `1bbb01c3…a92d6`), of which 442,929 came from real hardware rather than simulation, from 38,320 distinct vehicle identifiers. Uploaders may rate a log. A total of 590 logs carry an uploader crash rating (402 aircraft-caused,

188 pilot-caused); Table 2 details the 520 crash-rated logs from real hardware with duration $\geq$ 10 s and complete firmware version records. These uploader-assigned responsibility labels are not independently verified ground truth; they reflect the uploaders' own assessments. A notable discrepancy appears between uploader ratings and expert conclusions: while uploaders attributed 31.9% of rated crashes to the pilot (188 of 590), forum experts blamed the pilot in only 6.1% of all threads (9 of 148) and 8.6% of decided threads (9 of 105, Section 5.6). Across all 148 real crash threads, download attempts yielded 43 downloadable logs from 43 distinct threads (38 aircraft-fault and 5 pilot-fault; the remaining 105 had unhosted or expired external links). Cross-tabulating these 43 logs against archive ratings reveals that 9 carried uploader aircraft crash ratings (`crash_sw_hw`, with 100% expert agreement: all 9 diagnosed as aircraft faults), 33 were unrated in the archive index (28 aircraft, 5 pilot), 1 was rated unsatisfactory (aircraft), and exactly 0 were uploaded with a `crash_pilot` rating. End-users who crashed and sought community advice never self-identified pilot error on the forum. On the 14 overlapping forum threads whose logs carried an uploader aircraft crash rating in the index, experts agreed in 11 and were unclear in 3 (none blamed the pilot). This gap suggests potential politeness bias on public developer forums, selection bias in which incidents operators submit for community assistance, or a narrower definition of pilot error among autopilot developers compared to end users. Furthermore, forum diagnoses represent informed community opinion rather than independently verified accident inspection ground truth; original posters explicitly confirmed the diagnosis in only 47 of 94 aircraft threads and 5 of 9 pilot threads. Logs rated "good" or "great" served as normal flights. Logs were parsed with pyulog. We store derived features and log identifiers, not the logs.

A ULog file contains the parameters in force when logging started, any parameter changes during logging, timestamped messages published by the autopilot's modules, and logged text messages with a severity level. The analyses below use the vehicle status (arming state, navigation mode, failsafe flags), attitude and body-rate topics, rate setpoints, inertial-sensor status, actuator outputs, battery status, GPS position and the land detector. The vehicle identifier in the index is derived from the flight controller's hardware serial number; it identifies a flight controller, which in practice usually means one airframe, though a controller can be moved between frames.

Crash-rated logs differ from well-rated logs in composition (Table 2): they are shorter (median 54 s against 388 s, partly because a crash ends the log)

Table 2: Composition of crash-rated and well-rated logs from real hardware in the archive.

| | Crash-rated | Rated good or great |
|---|---|---|
| Logs | 520 | 21,112 |
| Median duration | 54 s | 388 s |
| Most frequent years | 2018 (91), 2025 (89), 2019 (87), 2017 (57) | 2018 (10,571), 2017 (5,748), 2019 (2,910) |
| Most frequent PX4 versions | 1.8 (70), 1.9 (66), 1.15 (65), 1.11 (65) | 1.7 (6,299), 1.8 (5,785), 1.6 (4,265) |
| Vehicle types | quadrotor 320, fixed wing 71, hexarotor 38 | quadrotor 19,890, fixed wing 620, hexarotor 291 |

and spread over more firmware generations. All between-group comparisons below therefore use either matching or within-drone designs, and we do not pool across firmware generations where measurements differ.

### *3.2. Samples*

Figure 1 and Table 3 show how each analysis sample was drawn. We evaluate two primary classification tasks: Task A (separating crash flights from normal flights) and Task B (binary attribution of crash responsibility between drone and pilot). We distinguish two distinct cohorts that each happen to number 148: (1) the leak-curve and attribution modeling sample of 148 crashes and 139 normal flights at $m = 0$ s (Section 4), and (2) the expert community sample of 148 crash discussion threads on the PX4 forum (Section 5.6). The attribution rule was evaluated on 205 crashes (93 development and 112 held-out). All 148 crashes entering the statistical attribution models are a strict complete-case subset of these 205 crashes (100% overlap); 57 crashes were excluded from modeling because required ULog topics were unrecorded or the flight window before cut-off was shorter than 10 s. In the leak-curve and attribution modeling sample, sample sizes across cut-off margins $m \in \{0, 2, 5, 10, 20, 30\}$ s reflect progressive attrition as flights with duration under 10 s before cut-off are dropped: for Task A (crash vs. normal), 148 vs. 139 ($m = 0$ s), 142 vs. 137 ($m = 2$ s), 135 vs. 133 ($m = 5$ s), 114 vs. 130 ($m = 10$ s), 100 vs. 128 ($m = 20$ s), and 89 vs. 124 ($m = 30$ s); for Task B (drone vs. pilot responsibility among crashes), 112 vs. 36 ($m = 0$ s, $n = 148$), 106 vs. 36 ($m = 2$ s, $n = 142$), 101 vs. 34 ($m = 5$ s, $n = 135$), 87 vs. 27 ($m = 10$ s, $n = 114$), 73 vs. 27 ($m = 20$ s, $n = 100$), and 63 vs. 26 ($m = 30$ s, $n = 89$). A constant complete-case subset of 213 flights (89

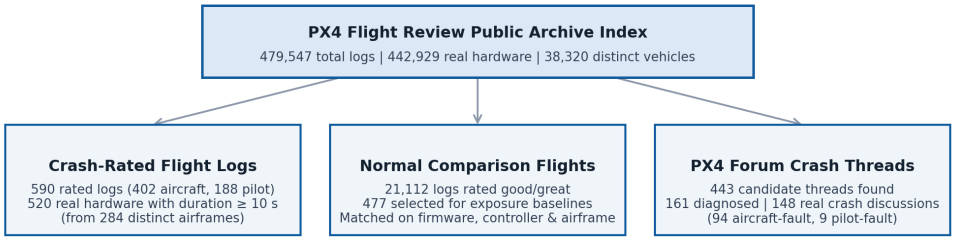
crashes and 124 normal flights; 63 drone and 26 pilot crashes) with pre-crash duration $\geq 40$ s was evaluated across all six margins without attrition.

Initial cross-sectional samples for Q1 and Q2 selected flights with PX4 1.10 or newer and at least 5 s of flight; normal flights were matched to crashes on vehicle type and drawn from vehicles without any crash rating. For the longitudinal within-drone and between-drone tests (Q3b, Q3c), only the eligible case count ($n = 12$), not any outcome or feature data, was inspected before deciding to lift the firmware restriction to avoid sample starvation; flights were instead strictly matched within each drone's firmware generation and measurement source. For the within-drone test every crash with a vehicle identifier and at least three earlier non-crash flights of 30 s or more was eligible (101 crashes; up to the five most recent earlier flights each). For the between-drone test each crashed drone was matched with up to two drones without any crash-rated log that shared the flight-controller hardware, the PX4 major.minor version and the vehicle type (66 matched sets, 124 control drones, three randomly chosen flights each, seed 7). Control drones were sampled without replacement, so no control served two cases. The three exposure flights of a crashed drone were its most recent non-crash flights dated on or before the crash; flights on the crash day were included in the declared test because the archive does not give their order within a day, and removed in a sensitivity analysis.

Not all selected flights enter every test. In the within-drone sample (101 crashes selected), 55 logs could not be read or had under 10 s before the cut-off; a drone enters a feature's test only if the crash flight and earlier flights have that feature from the same measurement source, and rate-tracking error is rotary-wing only and requires rate setpoints, leaving 61 analysed drones for vibration and 46 for tracking. In the between-drone sample (66 matched sets), 71 flights were unreadable or too short; a drone's value requires at least two readable flights, and case and controls must share the measurement source, leaving 60 analysed sets for vibration and 47 for tracking. In the between-drone sample, 124 matched control drones contributed 372 flights ($124 \times 3$), distinct from the 372 flights (67 crash flights and 305 earlier flights) evaluated in the within-drone predictive model.

### *3.3. Expert diagnoses*

We searched the PX4 discussion forum for threads that link a Flight Review log and mention a crash (443 threads, 410 with a log link). For the 161 threads with a reply from PX4 staff, the development team or a member




| Analysis Stream (Protocol) | Initial Candidate Pool | Exclusions & Attrition Criteria | Analyzed Sample Size |
|---|---|---|---|
| **Q1: Parameter Settings** | 100 crash, 477 normal | 43% crash logs lack factory default records | 57 crash, 113 normal |
| **Q2: Fixed Rule Attribution** | 205 crashes, 143 normal | None (evaluates all available crashes: 93 dev | 205 crashes, 143 normal |
| **Q2/Q3a: Data Leakage Curve** | 205 crashes, 143 normal | 57 dropped (missing topics or duration < 10 | 148 crash, 139 normal (m=0 s)<br>89 crash, 124 normal (m=30 s) |
| **Q3b: Within-Drone Longitudi** | 101 crashed drones | 34 dropped (single flight recorded or duratio | 67 multi-flight drones (372 flights)<br>61 vibration sets, 46 tracking sets |
| **Q3c: Between-Drone Matched** | 66 matched sets (101 case | 6 sets dropped (unmatched sensor metric ac | 60 vibration sets (124 controls, 372<br>47 rate-tracking sets |
| **Q4: Expert Community Cases** | 148 real crash threads | 105 logs unhosted or expired download links | 43 downloadable logs (38 aircraft, 5<br>37 aircraft logs with fault categorie |

Figure 1: From the public index to each analysis. Counts are logs unless stated.

at the forum's highest trust levels (148 of them describing a real crash), we recorded the expert's conclusion (aircraft, pilot, both, unclear, no diagnosis), the cause named, who gave it, whether the original poster confirmed it, and the sentence stating the diagnosis. Recording was AI-assisted and reviewed by an author; 123 of 124 recorded sentences were found verbatim in their thread by script. A second author then coded all threads blind to the first recording, with the same categories; a script confirmed that 150 of 151 coded rows quoted their own thread or recorded no diagnosis. On the 147 threads both coded, agreement (Cohen's kappa, 95% bootstrap interval) was 0.81 (0.71–0.88) for the expert's conclusion, 0.73 (0.65–0.81) for the cause and 0.79 (0.69–0.88) for whether the poster confirmed it, above the declared pass mark of 0.60. The two coders settled the 45 threads that differed on conclusion or cause by discussion, which changed 13 labels; counts and tests below use the settled labels, and the first recording led to the same conclusions (both are in the repository). The searches combined the Flight Review host names with the words crash, crashed, flyaway, fell, flipped and lost control, and

every returned thread was retrieved through the forum's public interface. A reply counted as expert when its author was marked as staff, belonged to the development team group or held one of the forum's two highest trust levels. Only public posts were used; usernames and post text are not redistributed.

## 4. Methods

### *4.1. Declared test protocols*

For each test, the selection, features, statistic, threshold and pass mark were committed and pushed to a public git repository before the data it used were downloaded or analysed. Unlike third-party timestamped registries (such as the Open Science Framework), commit timestamps in self-hosted git remain author-controlled until an independent snapshot is indexed; the git commit history establishes an immutable cryptographic SHA-1 chain in version control, and archived snapshots have been deposited in Software Heritage (`swh:1:rev:bad06ff1256865c7e63d44389aac174e0281291d`, `swh:1:rev:bbdd0420de4547ac7d67b84165292a46237b604d`, `swh:1:rev:624dd694e0d207dd9c44b1c9550faaf04453bc56`, and `swh:1:rev:3a9aa89417fa9ea2a2334e3a4794e2358fb713ee`) alongside the repository's git commit hashes, and will be minted into a Zenodo archive DOI upon publication. One amendment was data-dependent: after observing that the firmware filter of the within-drone test left only 12 eligible cases under the initial $\geq 1.10$ filter (without inspecting any outcome data), the firmware filter was removed and a fallback for older firmware was added. One coding error was corrected after a first run (logs whose CPU-load record lacked the load field were being skipped); no threshold changed. Secondary sensitivity analyses and diagnostic evaluations added during peer review are explicitly designated as exploratory and documented with commit hashes in Section 5.8.

### *4.2. Crash moment and analysis window*

For rotary-wing aircraft the crash moment $t_c$ is the first time after take-off at which the tilt $\theta = \arccos\left(1 - 2(q_x^2 + q_y^2)\right)$, computed from the attitude quaternion, exceeds 60°; for fixed-wing aircraft, and when no such tilt occurs, it is the end of the armed period. Normal flights use the end of the armed period. Take-off is the first sample after arming at which the land detector reports the vehicle airborne. An analysis with cut-off $m$ uses the window from take-off to $t_c - m$ and is skipped if the window is shorter than 10 s. Because normal flights include the complete descent, landing and disarm

sequence while crash windows terminate at impact or tilt, this introduces an inherent windowing asymmetry that contributes to differences in duration and whole-window aggregates.

### *4.3. Features*

Table 4 defines the features. Vibration is not measured the same way across firmware generations: newer logs report an accelerometer vibration metric, older ones an estimator vibration estimate. Comparisons are therefore only made between flights measured with the same source, and the source is recorded with every value. The rate-tracking error falls back to the attitude message's body rates when the angular-velocity topic is absent.

### *4.4. Tests*

*Q1 settings.* For logs that carry the firmware's record of default values, a protection was counted as disabled when its value differed from the default and switched it off (for example a failsafe action of "none"). Crash and normal logs were compared with Fisher's exact test.

*Q2 attribution.* Task B is defined as the binary attribution of crashes between pilot and aircraft fault (contrasted with Task A, which evaluates separating crash flights from normal flights). The declared rule labelled a crash "aircraft" if any of eight signs fired before $t_c$ (a critical battery warning or remaining charge below 10%; any motor at 95% or more for at least 1 s in the last 10 s; more than 50 clipping events; poor GPS in a GPS-dependent mode; a failure-detector flag; an error-level message; CPU load above 95%; an active failsafe), otherwise "pilot" if the mode at $t_c$ was pilot-flown, otherwise "unknown". The pass mark was a balanced accuracy of at least 0.70 on decided cases, answering at least 60% of crashes and being wrong on at most 20%. A model-based check used L2-regularised logistic regression on all features ($\lambda = 1$, standardised inputs, median imputation within training folds), evaluated by stratified five-fold cross-validation repeated five times and summarised by the ROC AUC [33]. As an exploratory sensitivity analysis, non-linear feature interactions were evaluated using depth-3 gradient-boosted decision trees.

The signs were chosen before the analysis to cover common failure modes of PX4 multirotors: power, propulsion, vibration, navigation, software and failsafes. The pilot rule was deliberately simple: if no aircraft sign fired and the vehicle was in a mode in which the pilot controls attitude or position

directly, the crash was attributed to the pilot. The rule was fixed before it was run on any crash log; only the list of message types available in the logs had been inspected.

*Q3a crash window.* The same model separated crash from normal flights at cut-offs $m \in \{0, 2, 5, 10, 20, 30\}$ s. We declared that the AUC would fall by at least 0.10 between $m = 0$ and $m = 10$ s if crash consequences drove it.

*Q3b and Q3c rank tests.* For a crashed drone $i$ with value $x_{i0}$ and comparison values $x_{i1}, \ldots, x_{in_i}$, the scaled rank is

$$r_i = \frac{1}{n_i} \sum_{j=1}^{n_i} \mathbf{1}\left[x_{ij} < x_{i0}\right], \tag{1}$$

which is 0 when the crashed drone has the lowest value and 1 when it has the highest; ties count against the hypothesis. A rank statistic was chosen for three reasons: the two vibration sources have different units and scales, so only orderings are comparable across drones; the number of comparisons differs between drones, which the exact null handles directly; and the distributions are skewed, with a few very rough flights, which would dominate a mean difference. In the within-drone test (Q3b) $x_{i0}$ is the crash flight, cut 10 s before $t_c$, and the comparisons are the same drone's earlier flights, each cut 10 s before the end of its armed period. In the between-drone test (Q3c) $x_{i0}$ is the median over the crashed drone's three most recent pre-crash flights and the comparisons are the medians of the matched controls. Under no difference each $r_i$ is uniform on $\{0, 1/n_i, \ldots, 1\}$ with mean 0.5, and $\bar{r}$ estimates the probability that the crashed drone exceeds a comparison, the quantity estimated by the Mann–Whitney statistic [34]. One-sided $p$-values come from 100,000 draws of this exact null; 95% confidence intervals from 10,000 bootstrap resamples of drones [35]. For the proportion of sets in which the crashed drone is the roughest, the expected null rate depends on the mix of set sizes: among the 60 analysed vibration sets, 35 had two matched controls (null probability 1/3) and 25 had one control (null probability 1/2), giving an exact Poisson-binomial expected null proportion of $(35 \times \frac{1}{3} + 25 \times \frac{1}{2})/60 = 40.28\%$ (null mean 24.167, SD 3.745). For duration adjustment in sensitivity analysis, log(vibration) was regressed on log(flight duration) within each measurement source across all exposure flights, and the rank test was repeated on the residuals. Each test had two primary features (vibration, rate-tracking error) at $\alpha = 0.025$.

*Q4 expert diagnoses.* Expert conclusions were compared with uploader labels. Fixed checks for the fault the expert named (tuning, vibration, GPS or compass, motor, battery) were declared useful if they fired on at least 50% of the logs with that diagnosis and at most 10% of normal flights, first with the full window and then with $m = 10\,\mathrm{s}$.

*Language-model panel.* The panel tested a design proposed for advisory diagnosis: several reviewers, each restricted to one slice of evidence (navigation, pilot inputs, GPS, battery, settings), vote on a fixed list of six causes and must cite numbered facts extracted from the log. Robustness was evaluated by replacing one reviewer with an adversary arguing for a chosen wrong cause, and planting hostile strings in uploader-controlled fields. Full details are provided in Appendix B.

*Multiplicity.* Seven confirmatory tests produce $p$-values. Holm's procedure [36] is reported over all seven in addition to each test's declared $\alpha$.

## 5. Results

### *5.1. Configuration settings (Q1)*

Of 57 crash logs and 113 normal logs carrying a firmware-default record, 29 (51%) and 52 (46%) had at least one protection disabled relative to the default (Fisher $p = 0.63$). Pilot-caused crashes had a protection disabled more often than aircraft-caused ones: 14 of 20 (70%) against 15 of 37 (41%), an observed difference of +29.5% (95% CI 3.9–55.0%, odds ratio 3.42, Fisher's exact two-sided $p = 0.052$). This contrast is exploratory, not part of the pre-declared confirmatory family, and does not reach statistical significance unadjusted (Fisher's exact two-sided $p = 0.052$). Furthermore, default parameter records were present only on newer firmware (missing for 43% of the 100 crash logs); on the 57 available cases, treating disabled protections as a pilot-attribution classifier yields a sensitivity of 70.0%, a specificity of 59.5%, and a balanced accuracy of 0.647, below the declared 0.70 pass mark. None of 100 crash logs changed a safety-critical or flight-limit setting during the flight.

### *5.2. Attributing a crash to the pilot or the aircraft (Q2)*

The fixed rule reached a balanced accuracy of 0.57 on 93 development crashes (35% wrong) and 0.47 on 112 held-out crashes (47% wrong), against

the declared 0.70, and labelled 39% of 143 normal flights as aircraft faults (Table 5). The signs fired at similar rates whichever party the uploader blamed (Fig. 2): motor saturation in 23% of aircraft-caused and 29% of pilot-caused crashes (1% of normal flights), battery warnings in 15% and 18% (3%). Error messages and failsafes were no more frequent in crashes than in normal flights. Two signals noticed after the development run did not replicate on held-out crashes: accelerometer clipping (24% against 8%, $p = 0.045$) and manual mode at the crash (18% against 10%, $p = 0.165$).

In Task B, sample sizes decline across margins as flights under 10 s are excluded: 112 drone and 36 pilot crashes at $m = 0$ s ($n = 148$), 106 vs. 36 at $m = 2$ s ($n = 142$), 101 vs. 34 at $m = 5$ s ($n = 135$), 87 vs. 27 at $m = 10$ s ($n = 114$), 73 vs. 27 at $m = 20$ s ($n = 100$), and 63 vs. 26 at $m = 30$ s ($n = 89$). On the full sample ($n = 148$ at $m = 0$ s, 75.7% drone-blamed), cross-validated logistic regression yielded an ROC AUC of 0.408 (bootstrap SE 0.062, 95% CI 0.288–0.530, upper 95% bound 0.510) at $m = 0$ s, 0.422 (SE 0.072, 95% CI 0.283–0.566, upper bound 0.542) at $m = 10$ s ($n = 114$), and 0.501 (SE 0.073, 95% CI 0.358–0.645, upper bound 0.619) at $m = 30$ s ($n = 89$). Evaluated on the constant complete-case sample of 89 crashes (63 drone, 26 pilot; 70.8% drone-blamed) plotted in Fig. 3, cross-validated logistic regression yielded an AUC of 0.466 (bootstrap SE 0.072, 95% CI 0.325–0.607, upper bound 0.584) at $m = 0$ s, 0.489 (SE 0.074, 95% CI 0.343–0.633, upper bound 0.611) at $m = 10$ s, and 0.501 (SE 0.073, 95% CI 0.358–0.645, upper bound 0.619) at $m = 30$ s. (Repeated cross-validation folds showed nominal mean fold AUCs of $0.450 \pm 0.015$ at $m = 0$ s, $0.449 \pm 0.022$ at $m = 10$ s, and $0.458 \pm 0.034$ at $m = 30$ s on this subset.)

To calibrate whether these below-chance AUCs represent an inverted physical signal or finite-sample noise, we computed a 1,000-iteration permutation null distribution on Task B by randomly shuffling crash attribution labels across folds: the null distribution had mean 0.497 (SD 0.090) with an empirical 95% interval of 0.313–0.660, establishing a 95% detection threshold of approximately 0.660. The observed cross-validated AUC of 0.449 falls comfortably within this chance range (two-sided permutation $p = 0.509$). Crucially, because only 36 crashes carry pilot labels, bootstrap resampling across crashes demonstrates that the upper 95% confidence bounds on observed models (0.510 on the full sample, 0.584 on the constant subset at $m = 0$ s; 0.542 and 0.611 at $m = 10$ s) lie safely below the 0.660 detection threshold, confirming that observed performance cannot be distinguished from chance and rules out models achieving detection beyond noise.

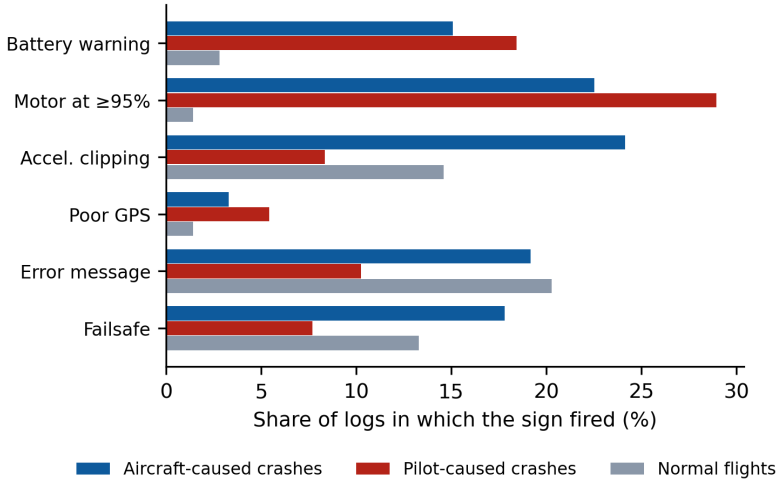



Figure 2: How often each aircraft-fault sign fired in held-out crashes, by uploader label, and in normal flights.

To evaluate whether non-linear feature interactions could extract subtle pre-crash signatures, we trained gradient-boosted decision tree ensembles on Task B. Depth-1 decision stumps reached an AUC of 0.430 at $m = 0$ s (bootstrap SE 0.062, 95% CI 0.310–0.551, upper bound 0.534), falling to 0.413 at $m = 10$ s (SE 0.069, 95% CI 0.281–0.548, upper bound 0.528). Depth-3 boosted trees reached an AUC of $0.595 \pm 0.130$ at $m = 0$ s ($0.604 \pm 0.095$ at $m = 2$ s, $0.621 \pm 0.105$ at $m = 5$ s), falling to $0.553 \pm 0.171$ at $m = 10$ s, $0.579 \pm 0.150$ at $m = 20$ s, and $0.548 \pm 0.104$ at $m = 30$ s. Across all cut-offs, the upper 95% confidence bounds of the boosted ensembles remain below the 0.660 permutation null detection threshold. Modeling non-linear interactions does not rescue crash attribution; once the impact window is excluded, classification remains near chance. We conclude that with these rules, models and labels, automatic responsibility attribution is not achievable.

### *5.3. The crash window (Q3a)*

In the primary declared sample, crash flights were separated from normal flights with a cross-validated AUC of 0.88 (SD 0.01) at $m = 0$ s ($n = 148$ crashes, 139 normal flights), 0.88 at $m = 2$ s (142 vs. 137), 0.88 at $m = 5$ s (135 vs. 133), 0.87 (SD 0.01) at $m = 10$ s (114 vs. 130), 0.86 at $m = 20$ s (100 vs. 128), declining to 0.86 at $m = 30$ s (89 vs. 124); the declared drop of 0.10 was not observed (observed drop of 0.01). To evaluate whether this apparent

stability was an artefact of sample attrition as shorter flights dropped out at larger margins, we evaluated the cross-validation on the fixed constant subset of 213 flights with duration $\geq 40\,\mathrm{s}$ (89 crashes, 124 normal flights; plotted in Fig. 3) across all six margins. On this constant sample, model AUC declined smoothly from 0.895 (SD 0.007) at $m = 0\,\mathrm{s}$ to 0.876 at $m = 2\,\mathrm{s}$, 0.873 at $m = 5\,\mathrm{s}$, 0.868 at $m = 10\,\mathrm{s}$ (a drop of 0.027), 0.862 at $m = 20\,\mathrm{s}$, and 0.864 at $m = 30\,\mathrm{s}$. The absence of a large drop in Task A AUC (0.895 to 0.868) indicates that the cross-sectional model does not rely primarily on transient crash-impact telemetry. Instead, classification is likely influenced by flight envelope differences and airframe characteristics that correlate with duration (Table 6). Although flight duration was deliberately excluded from the 11 normalized model features, duration alone separates crash from normal flights with an AUC of 0.662 at $m = 0\,\mathrm{s}$ (crash median 71.0 s vs. normal median 169.0 s) and 0.634 at $m = 10\,\mathrm{s}$, and correlates significantly with several normalized features: longer flights exhibit larger 95th-percentile tilt ($\rho = +0.470$ at $m = 0\,\mathrm{s}$, $+0.484$ at $m = 10\,\mathrm{s}$), greater voltage sag ($\rho = +0.321$ and $+0.357$), and larger stick deflections ($\rho = +0.363$ and $+0.233$), while shorter flights show higher rate-tracking errors ($\rho = -0.248$ at $m = 0\,\mathrm{s}$). The absence of a large collapse holds on identical flights, confirming that changing sample composition across cut-offs did not mask a collapse.

Leakage was nevertheless present for individual physical checks. The rate-tracking feature alone fell from an AUC of 0.81 to 0.69 as the crash was excluded, while vibration stayed between 0.74 and 0.77. A fixed tuning check that found 10 of 12 candidate expert-diagnosed tuning faults (7 of 9 on paired surviving logs) with the crash included found only 1 of 9 once the last 10 s were removed (Section 5.6).

Crucially, the high cross-sectional model separation (AUC 0.88) does not demonstrate unconfounded anticipation of an impending crash: crash and normal flights differ fundamentally in duration (median 54 s against 388 s in the broader archive; 71 s vs. 169 s in the complete-case sample), firmware era distribution, and operational flight envelope. Table 6 gives the single-feature AUCs. Values below 0.5 indicate features that are lower in crash flights: crash flights reached lower tilt and smaller stick deflections over the whole flight than normal flights, consistent with short, aborted or cautious testing flights. When the exact same 11-feature model was evaluated *within the same drone*—separating each crashed drone's pre-crash window (cut 10 s before impact) from its own earlier flights across 67 multi-flight drones (372 flights: 67 pre-crash flights and 305 earlier flights, distinct from the 372 between-

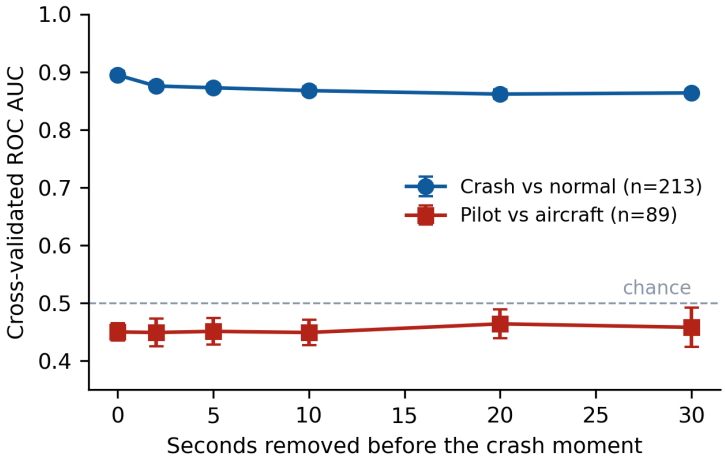



Figure 3: Cross-validated AUC as the end of the analysis window moves away from the crash moment. Bars: standard deviation over five repeats of five-fold cross-validation.

drone control flights)—the pooled Group-CV AUC fell from 0.88 to 0.580 (95% bootstrap CI 0.528–0.633), and the mean within-drone pairwise AUC was 0.605 (95% bootstrap CI 0.516–0.692). Matching on the airframe substantially attenuates the cross-sectional separation from AUC 0.88 to AUC 0.580, demonstrating that cross-sectional models reflect airframe types, operator styles, and duration confounds rather than detecting impending failure, in agreement with the within-drone rank test ($\bar{r} = 0.48$).

### *5.4. Within the same drone and between drones (Q3b, Q3c)*

Relative to the same drone's earlier flights, the crash flight was neither more vibrating ($\bar{r} = 0.48$, 95% CI 0.38–0.58, $p = 0.67$, 61 drones) nor worse at tracking its commands ($\bar{r} = 0.50$, 0.38–0.63, $p = 0.47$, 46 drones; Fig. 4). Presented as an exclusion bound, the upper 95% confidence bound of 0.58 rules out pre-crash vibration surges within the same drone of the magnitude observed between drones ($\bar{r} = 0.67$). Relative to matched drones without a recorded crash, drones that later crashed vibrated more on their ordinary pre-crash flights ($\bar{r} = 0.67$, 0.57–0.77, $p = 0.0023$, 60 sets; Holm-adjusted $p = 0.016$), while the tracking error did not differ significantly ($\bar{r} = 0.57$, 0.44–0.70, $p = 0.15$, 47 sets). The vibration difference was clearest for aircraft-caused crashes ($\bar{r} = 0.67$, $p = 0.005$, 46 sets) and similar in size but not significant for pilot-caused ones ($\bar{r} = 0.64$, $p = 0.16$, 14 sets).

As a screening signal, the crashed drone was the roughest of its matched set in 33 of 60 sets (55.0%, Wilson 95% CI 42.5–66.9%, against 40.3% expected under the exact Poisson-binomial null; exact one-sided Poisson-binomial $p = 0.0137$). Furthermore, in 38 of 60 sets (63.3%, Wilson 95% CI 50.7–74.4%), the crashed drone exceeded the third quartile (Q3) of its matched controls. Sensitivity differed between measurement sources: 32 of 48 crashed drones (66.7%, Wilson 95% CI 52.5–78.3%) exceeded control Q3 on the older estimator metric, compared to 6 of 12 (50.0%, Wilson 95% CI 25.4–74.6%) on the newer accelerometer metric.

In practical fleet operations, positive predictive value (PPV) depends heavily on the operational flag rate and the fleet base rate. In the archive, 284 distinct vehicles accounted for all 520 real-hardware crash-rated logs out of 38,320 distinct vehicle identifiers, establishing an empirical vehicle crash base rate of 0.741% (284/38,320). When restricted to screenable multi-flight vehicles (airframes with $\geq 2$ recorded flights, the operational prerequisite for pre-crash longitudinal screening), the crash base rate is 0.689% (182 crashed vehicles out of 26,421 multi-flight airframes). We emphasize that these base rates represent the lifetime probability of an airframe ever receiving a crash rating in the archive index, rather than an operational flight-hour incident rate, because the public archive does not record flight hours per airframe. Reported confidence intervals for PPV reflect sampling uncertainty in sensitivity (derived via 10,000 bootstrap resamples over crashed cases); the operational base rate is treated as a fixed fleet parameter. We evaluate screening performance under two distinct operational paradigms:

*Empirical matched-set screening (45.1% flag rate):* In our matched sets, flagging drones that exceed the upper quartile of their matched controls yields 63.3% sensitivity (38 of 60). Because sets contain 1 or 2 controls, the weighted null flag rate is 45.1%. At the 0.741% vehicle base rate, this yields an operational PPV of 1.04% (95% bootstrap CI 0.83–1.25%), an unflagged risk of 0.496%, a relative risk of 2.09× over unflagged airframes, a lift of 1.40× over base rate, and 96.4 inspections per detected crash (PPV 0.96%, 104 inspections per crash at the 0.689% screenable base rate).

*Fleet-wide reference screening:* Rather than comparing solely within small local sets, an operator can screen airframes against an established reference distribution of non-crash control drones sharing the same flight controller hardware, firmware generation, and measurement source (20 strata, 107 unique control drones; overall AUC / mean percentile 0.689, 95% CI 0.619–0.758). Each airframe is ranked against control flights within its matching

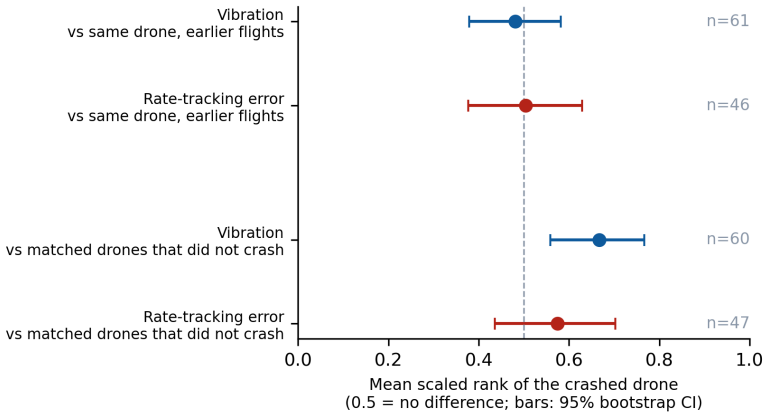



Figure 4: Mean scaled rank of the crashed drone against its own earlier flights (top) and against matched drones without a recorded crash (bottom). 0.5 indicates no difference.

hardware and sensor stratum. At a 25% flag rate, this stratum-matched screening identifies 50.0% of crashed airframes (30 of 60, 95% CI 36.7–63.3%), yielding an operational PPV of 1.48% (95% CI 1.09–1.88%), an unflagged risk of 0.493%, a relative risk of 3.00×, a lift of 2.00× over the 0.741% base rate, and 67.5 inspections per detected crash (at the 0.689% screenable base rate: PPV 1.37%, 95% CI 1.01–1.74%, lift 1.99×, 73.1 inspections per crash). Tightening the threshold to the top decile (10% flag rate) achieves 33.3% sensitivity (20 of 60, 95% CI 21.7–45.0%), an operational PPV of 2.47% (95% CI 1.61–3.33%), an unflagged risk of 0.549%, a relative risk of 4.50×, a lift of 3.33×, and 40.5 inspections per detected crash (at the 0.689% screenable base rate: PPV 2.26%, 95% CI 1.48–3.06%, lift 3.28×, 44.2 inspections per crash). Under a broader source-matched reference (matching on measurement source alone; AUC 0.655, 95% CI 0.591–0.716), a 25% flag rate yields 40.0% sensitivity, PPV 1.19% (lift 1.60×, 84.3 inspections per crash), rising to 25.0% sensitivity, PPV 1.85% (lift 2.50×, 54.0 inspections per crash) at the 10% flag rate. In all scenarios, vibration screening functions as a broad maintenance inspection filter rather than an operational crash warning. screening functions as a broad maintenance inspection filter rather than an operational crash warning.

### *5.5. Robustness of the between-drone result (exploratory)*

The vibration result held when controls were restricted to the same airframe as the case ($\bar{r} = 0.69$, $p = 0.008$, 39 sets), when case flights from the crash day were removed ($\bar{r} = 0.76$, $p = 0.003$, 25 sets) and after adjusting for flight duration ($\bar{r} = 0.69$, $p = 0.0005$, 57 sets; Fig. 5). Duration was barely correlated with vibration (Spearman $\rho = 0.05$ and $-0.04$ for the two measurement sources), although case flights were longer than control flights (median 289 s against 173 s). The sharp sample shrinkage when removing crash-day flights (from 60 to 25 sets) indicates that many pre-crash flights in the public archive occurred during the same flight session or day. The archive index did not record intra-day flight sequence, though parsing raw GPS timestamps in future work could establish whether crash-day flights immediately preceded the crash. This points to potential confounding by development or tuning sessions rather than progressive operational degradation: the "maintenance view" (vibration reflecting mechanical wear over time) competes directly with an "experimental airframe" view (poorly tuned or prototype drones vibrate more and crash during testing). Furthermore, matching was limited to flight controller hardware, PX4 major.minor version and vehicle type; it did not control for propellers, payload, airframe size, or operator identity, so controls from the same operator were not clustered.

The vibration difference was carried by the older estimator metric ($\bar{r} = 0.68$, $p = 0.005$, 48 sets); within the original cohort, the exploratory subset with the newer accelerometer metric showed a smaller, non-significant difference ($\bar{r} = 0.62$, $p = 0.191$, 12 sets; Fig. 5). A declared replication protocol on newer accelerometer logs selected 26 sets, yielding 19 usable matched sets (9 sets pooled from the initial cohort and 10 new cases entirely outside the original test). On the 10 new cases, the mean scaled rank was $\bar{r} = 0.50$ (95% CI 0.20–0.75, $p = 0.5760$), which is inconclusive given the wide confidence interval. Pooling all 19 usable sets gives $\bar{r} = 0.58$ (95% CI 0.39–0.76, one-sided $p = 0.251$). Statistical design power at $N = 19$ for an assumed effect of $\bar{r} = 0.67$ was 0.379 (and 0.118 if the true effect is $\bar{r} = 0.58$). Statistical sample sizing at one-sided $\alpha = 0.025$ indicates that detecting $\bar{r} = 0.67$ ($\Delta = 0.17$) with 80% power requires 55 matched sets; however, this assumes the discovery effect size was not inflated by winner's curse. If the true effect size is closer to the pooled estimate of $\bar{r} = 0.58$ ($\Delta = 0.08$), 247 sets are required. Because 247 matched sets exceed the capacity of the current single archive, resolving this effect definitively requires prospective multi-fleet partner deployments.

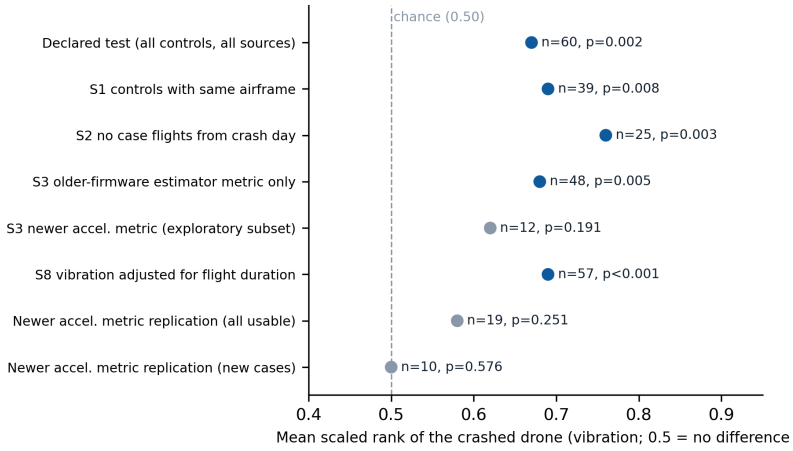



Figure 5: Between-drone vibration result under alternative specifications (exploratory). Blue: $p < 0.025$.

### *5.6. Expert diagnoses (Q4)*

Of 148 forum threads describing a real crash, the expert attributed it to the aircraft in 94 (64%), to the pilot in 9 (6%), to both in 2, and was unclear or gave no diagnosis in 43 (29%). Tuning or configuration was the cause named most often (26 of 94; 14 of the 47 confirmed by the poster), followed by GPS, compass or estimator problems (18) and motors, ESCs or propellers (16) (Fig. 6). Of 14 threads whose log carried an aircraft crash rating, the expert agreed in 11 and was unclear in 3; none attributed the crash to the pilot. Download attempts across all 148 threads yielded 43 downloadable logs from 43 distinct threads (38 aircraft-fault and 5 pilot-fault; the remaining 105 had unhosted or expired external links). Of the 38 aircraft logs, exactly 37 were associated with checkable fault categories in Table 7.

Table 7 compares the fault checks with the full flight window ($m = 0\,\mathrm{s}$) and with the last 10 s before the crash removed ($m = 10\,\mathrm{s}$). With the window ending 10 s before the crash, no check met the declared criterion. The tuning check illustrates the diagnostic leakage described in Section 5.3: across all candidate logs with the full window, the check fired on 10 of 12 (83.3%). Across the 9 constant tuning logs with duration $\geq 10\,\mathrm{s}$ surviving the 10 s cut-off without sample attrition, firing dropped from 7 of 9 (77.8%) with the full window to 1 of 9 (11.1%) at $m = 10\,\mathrm{s}$ (6 logs lost firing, 0 gained; exact two-sided McNemar $p = 0.03125$). However, this exploratory finding

is descriptive rather than confirmatory: across the five diagnostic checks evaluated, an unadjusted $p = 0.031$ does not survive Bonferroni multiplicity correction ($p \approx 0.16$). Furthermore, the other four checks exhibited low baseline sensitivity even on full windows ($\leq 25\%$, with 0/4 for motor and 0/2 for battery on paired logs), meaning they had virtually no baseline firing to lose, while tuning exceeded the declared 10% normal-flight false-alarm ceiling (13%). Thus, while rate-tracking error demonstrated marked leakage, claims that leakage is strictly confined to tuning must be tempered by the limited baseline sensitivity of the remaining checks.

To test diagnostic attribution on independently evaluated incidents, we evaluated the rule across all 43 downloadable logs behind expert forum threads (38 aircraft-fault and 5 pilot-fault). As noted in Section 3, cross-tabulating these 43 logs against archive ratings revealed that all 9 logs carrying an uploader rating were labeled `crash_sw_hw` (and all 9 were diagnosed as aircraft faults by experts, 100% agreement); the 5 expert pilot crashes were submitted unrated, indicating that operators seeking community assistance did not self-incriminate as pilot error. Evaluated on the 38 aircraft incidents, the fixed rule predicted "aircraft" in 18 (47.4%), "unknown" in 6 (15.8%), and erroneously blamed the "pilot" in 14 (36.8%). On decided aircraft cases ($n = 32$), the rule was wrong on 43.8% (14/32). Detailed inspection of flight mode sequences confirmed the failure mechanism: in all 14 misclassified logs, the vehicle was in a pilot-flown mode at the crash moment (`POSCTL` in 7 logs, `STAB` in 4, `ALTCTL` in 3; including log `e9034973`, which transitioned `POSCTL` $\rightarrow$ `AUTO_4` $\rightarrow$ `POSCTL` as the pilot attempted manual recovery). Because no hardware failsafe fired before impact, the rule defaulted to blaming the pilot.

Conversely, when evaluated on the 5 pilot-fault incidents, the fixed rule predicted "aircraft" in 4 (80.0%) and "pilot" in only 1 (20.0%; pilot recall 1 of 5, Wilson 95% CI 3.7–71.0%), misclassifying 4 of 5 pilot crashes because motor saturation, accelerometer clipping, or failsafe activation occurred as the vehicle struck the ground. Combining both classes across all 37 decided expert logs (32 aircraft, 5 pilot), the rule produced 18 incorrect decisions out of 37 (48.6% error rate), yielding a balanced accuracy of $(56.25\% + 20.0\%)/2 = 38.1\%$ (95% bootstrap CI 0.20–0.59), which does not differ significantly from chance (0.50). Similarly, when evaluated out-of-fold on the 30 aircraft logs with complete telemetry features, the logistic regression model predicted "aircraft" in 27 (90.0%) and "pilot" in 3 (10.0%), defaulting strongly toward its aircraft training prior (72.3% average across 5-fold CV training folds; 75.7% on the full 148-crash set, 112/148). We emphasize the

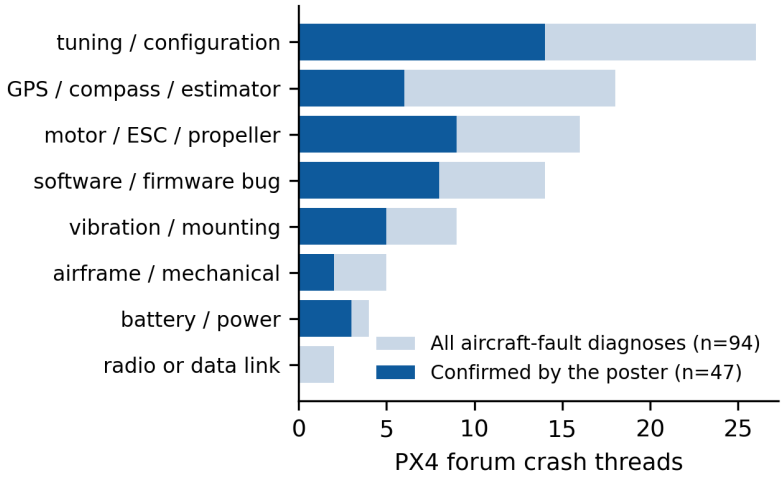



Figure 6: Causes named by experts in PX4 forum threads that attributed a crash to the aircraft.

caveat that this evaluation measures agreement between a heuristic rule and expert community reading of the same telemetry, rather than ground truth: original posters explicitly confirmed the diagnosis in only 47 of 94 aircraft-fault threads (50.0%) and 5 of 9 pilot-fault threads (55.6%). Nonetheless, this two-class comparison illustrates the challenge of rule-based attribution: the heuristic rule disagreed with expert community consensus on nearly half of decided cases (48.6%), failing to demonstrate agreement above chance.

### *5.7. Language-model review panels (summary)*

Because language-model assistants are increasingly proposed for log review, we conducted a controlled exploratory evaluation of local models (3–4B parameters) configured as single agents and multi-agent review panels on 60 author-generated fault cases and worst-case adversarial prompts. As detailed in Appendix B, the fixed rule reached 75% right and 0% wrong (referring 25% to humans), whereas local language models produced substantial error rates (10–63% wrong), suffered from mode collapse or abstention on active causes, and proved vulnerable to consensus manipulation by a single compromised reviewer (flipping 45–56 of 60 cases). In contrast, deterministic numeric telemetry extraction prevented direct prompt injection by construction.

### *5.8. Analyses added during revision (exploratory)*

To preserve the boundary between pre-declared confirmatory hypotheses and exploratory post-hoc investigations [17], Table 8 documents the analyses added during peer review, their motivation, key results, and the git commit hashes where their declarations or implementations were introduced. None of these exploratory additions altered the conclusions of the pre-declared tests; rather, they provided empirical bounds on confounding, evaluated alternative non-linear architectures, and quantified operational screening performance.

Table 3: Analysis samples and evaluation cohorts.

| Sample | Crashes | Comparison | Question |
|---|---|---|---|
| Settings | 100 (57 with default records) | 477 normal (113 with default records) | Q1 |
| Development crashes | 93 analysed | – | Q2 rule |
| Held-out crashes | 112 analysed | 143 normal (full logs) | Q2 replication |
| Leak-curve set | 148 at $m = 0\,\mathrm{s}$ (89 at $m = 30\,\mathrm{s}$) | 139 normal (124 at $m = 30\,\mathrm{s}$) | Q2 model, Q3a |
| Constant-subset leak curve | 89 crashes ($\geq 40\,\mathrm{s}$ duration) | 124 normal ($n = 213$ constant across margins) | Q2 model, Q3a |
| Within-drone rank test | 101 selected (61 and 46 analysed) | 475 earlier flights of the same drones | Q3b |
| Within-drone predictive model | 67 multi-flight drones (67 pre-crash) | 305 earlier flights of same drones ($n = 372$) | Q3b |
| Between-drone matched sets | 66 matched sets (60 and 47 analysed) | 124 matched drones, 372 flights | Q3c |
| Expert forum cases | 148 real crash threads | 43 logs downloadable (38 aircraft, 5 pilot) | Q4 |

Table 4: Features computed on each analysis window.

| Feature | Definition |
|---|---|
| Rate-tracking error | RMS of commanded minus measured body rate, roll and pitch ($\mathrm{rad\,s^{-1}}$; rotary wing) |
| Tilt (p95) | 95th percentile of tilt angle (deg) |
| Vibration (p95) | 95th percentile of the accelerometer vibration metric, or of the estimator vibration estimate on older firmware |
| Clipping rate | accelerometer clipping events per second |
| Motor saturation | share of time with any motor output at 95% or more (rotary wing) |
| Battery minimum | lowest reported remaining charge (%) |
| Voltage sag | (start voltage − minimum voltage) / start voltage |
| Poor GPS | share of time with horizontal accuracy above 5 m or fewer than 6 satellites |
| Failsafe | 1 if a failsafe was active at any time |
| Manual share | share of time in a pilot-flown mode (manual, altitude, position, acro, stabilised) |
| Stick deflection (p95) | 95th percentile of the larger roll or pitch stick deflection |

Table 5: Confirmatory tests, their declared pass marks and outcomes. Holm adjustment over the seven $p$-valued tests.

| Test (declared criterion) | Result | Holm $p$ | Outcome |
|---|---|---|---|
| Protections disabled, crash vs normal | 51% vs 46%, $p = 0.63$ | 1.00 | No difference |
| Attribution rule, dev. (BA $\geq 0.70$) | BA 0.57, 35% wrong | – | Failed |
| Attribution rule, held-out | BA 0.47, 47% wrong | – | Failed |
| Clipping, aircraft vs pilot ($p < 0.025$) | 24% vs 8%, $p = 0.045$ | 0.27 | Not supported |
| Manual mode, pilot vs aircraft ($p < 0.025$) | 18% vs 10%, $p = 0.165$ | 0.73 | Not supported |
| AUC drop from 0 to 10 s ($\geq 0.10$) | 0.88 to 0.87 | – | Not supported |
| Within-drone vibration ($p < 0.025$) | $\bar{r} = 0.48$, $p = 0.67$ | 1.00 | Not supported |
| Within-drone tracking ($p < 0.025$) | $\bar{r} = 0.50$, $p = 0.47$ | 1.00 | Not supported |
| Between-drone vibration ($p < 0.025$) | $\bar{r} = 0.67$, $p = 0.0023$ | 0.016 | Supported on older metric only; replication inconclusive |
| Between-drone tracking ($p < 0.025$) | $\bar{r} = 0.57$, $p = 0.15$ | 0.73 | Not supported |
| Expert-fault checks, $m =$ 10 s ($\geq 50\%$, $\leq 10\%$) | best: vibration 3/5 vs 11% | – | None met |

Table 6: Single-feature AUC for separating crash from normal flights at three cut-offs (no fitting; 0.5 is chance; values below 0.5 mean lower in crash flights).

| Feature | $m = 0\,\mathrm{s}$ | $m = 10\,\mathrm{s}$ | $m = 30\,\mathrm{s}$ |
|---|---|---|---|
| Rate-tracking error | 0.81 | 0.72 | 0.69 |
| Vibration (p95) | 0.74 | 0.72 | 0.77 |
| Manual share | 0.67 | 0.68 | 0.69 |
| Motor saturation | 0.67 | 0.66 | 0.65 |
| Clipping rate | 0.59 | 0.54 | 0.55 |
| Voltage sag | 0.55 | 0.53 | 0.48 |
| Poor GPS | 0.53 | 0.55 | 0.54 |
| Failsafe | 0.50 | 0.47 | 0.47 |
| Battery minimum | 0.42 | 0.40 | 0.38 |
| Tilt (p95) | 0.41 | 0.36 | 0.33 |
| Stick deflection (p95) | 0.32 | 0.33 | 0.35 |

Table 7: Fault checks on logs whose expert named the matching fault, with the full window ($m = 0\,\mathrm{s}$) and with the last 10 s before the crash removed ($m = 10\,\mathrm{s}$). Declared criterion: at least 50% of diagnosed logs and at most 10% of normal flights. Paired columns show constant logs surviving across both windows.

| | Diagnosed ($m = 0\,\mathrm{s}$) | | Diagnosed | Paired $\Delta$ | Normal comparison | |
|---|---|---|---|---|---|---|
| Expert-named fault | All logs | Paired | $m = 10\,\mathrm{s}$ | ($p$-value) | $m = 0\,\mathrm{s}$ | $m = 10\,\mathrm{s}$ |
| Tuning or configuration | 10/12 (83%) | 7/9 (78%) | 1/9 (11%) | $-6$ ($p = 0.031$) | 18/142 (13%) | 7/130 (5%) |
| Vibration or mounting | 3/5 (60%) | 3/5 (60%) | 3/5 (60%) | 0 ($p = 1.00$) | 7/48 (15%) | 4/38 (11%) |
| GPS, compass or estimator | 1/5 (20%) | 1/4 (25%) | 1/4 (25%) | 0 ($p = 1.00$) | 10/143 (7%) | 5/130 (4%) |
| Motor, ESC or propeller | 1/9 (11%) | 0/4 | 0/4 | 0 ($p = 1.00$) | 8/143 (6%) | 5/130 (4%) |
| Battery or power | 0/4 | 0/2 | 0/2 | 0 ($p = 1.00$) | 4/143 (3%) | 1/130 (1%) |

Table 8: Exploratory analyses added during peer review, their motivation, key results, and git commit hashes. Archived snapshots are indexed in Software Heritage (`swh:1:rev:bad06ff1256865c7e63d44389aac174e0281291d`, `swh:1:rev:bbdd0420de4547ac7d67b84165292a46237b604d`, `swh:1:rev:624dd694e0d207dd9c44b1c9550faaf04453bc56`, and `swh:1:rev:3a9aa89417fa9ea2a2334e3a4794e2358fb713ee`) and will be minted into a Zenodo DOI.

| Analysis | Motivation | Result | Commit |
|---|---|---|---|
| 1,000-permutation null (Task B) | Calibrate detection threshold | Null mean 0.497, 95% threshold 0.660; observed AUC upper bounds 0.510–0.584 | `bad06ff` |
| Boosted decision trees (Task B) | Test non-linear feature interactions | Stumps AUC $0.430 \rightarrow 0.413$; Depth-3 $0.595 \rightarrow 0.553$ at $m = 10\,\mathrm{s}$ | `bad06ff` |
| Constant-subset leak curve | Exclude sample attrition ($n = 213$) | AUC declines smoothly $0.895 \rightarrow 0.868$ ($m = 10\,\mathrm{s}$) | `bbdd042` |
| Paired diagnostic leakage test | Test leakage on constant logs ($n = 9$) | Tuning check $7/9 \rightarrow 1/9$, exact McNemar $p = 0.03125$ | `bbdd042` |
| Within-drone predictive model | Account for airframe confounding | Group-CV AUC 0.580; mean within-drone AUC 0.605 | `bad06ff` |
| Poisson-binomial set null | Account for mixed set sizes ($n = 60$) | Exact null rate 40.28%; observed 55.0% ($p = 0.0137$) | `bad06ff` |
| Screening PPV and CIs | Quantify operational fleet value | Screenable base rate 0.689% (PPV 1.37–2.26%); fleet base rate 0.741% (PPV 1.48–2.47%) | `bad06ff` |
| Sensitivity by vibration source | Compare older vs. newer metrics | Older: 32/48 (66.7%); newer: 6/12 (50.0%) | `bad06ff` |
| 43 expert crash logs attribution | Evaluate against independent causes | Two-class test: BA 38.1% (95% CI 0.20–0.59); pilot recall 1/5; uploader | `bad06ff` |

## 6. Design implications: an assurance tool built on the findings

The authors develop AeroFleet 2.0, an open-source, ground-side assurance tool for PX4 and ArduPilot fleets. It reads the files an operator already has (parameter files, mission files and flight logs), contacts no aircraft and calls no language model. This section reports design consequences and engineering checks; the paper's empirical evidence is presented in Sections 3–5. Table 9 links each result above to a design decision and to the check that was made of it. AeroFleet 2.0's checks are primarily engineering validation tests, not evidence of improved operational safety; demonstrating safety benefits in field operations requires prospective fleet evaluation and user studies, which remain future work (Section 7). The injection, search, and replay validation checks are detailed in Appendix A.

*Evidence, not verdicts.* A flight review reads an ArduPilot or PX4 log and the approved mission file and lists, in time order, mode changes (with a pilot taking over from an automatic mode flagged), settings changed in flight, battery and voltage warnings, poor GPS and deviations from the plan. It reports the quality of that evidence (sample rate, satellite count and dilution of precision, height reference, clock source) so that a reader can judge how far to trust it, and leaves the conclusion to a person. Given the attribution results, a tool that issued a pilot-or-aircraft verdict would claim more than the log supports.

*Ranking drones, not predicting flights.* Each reviewed flight's vibration is stored with its measurement source; drones are ranked against others measured the same way. The top decile (10%) is prioritized for mechanical inspection (with the top quartile, 25%, as an alternative operational threshold). In the archive, this ranking identified the drone that later crashed as the roughest in 55.0% of matched sets (against 40.3% expected under the exact Poisson-binomial null, $p = 0.014$; 1.37× lift over the null rate; relative risk 3.58× over unflagged airframes), and 63.3% exceeded the control third quartile. In operational terms, under a stratum-matched fleet-wide reference distribution at the 0.689% screenable base rate (and 0.741% fleet base rate), prioritizing the top decile (10%) achieves an operational PPV of 2.26–2.47% (relative risk 4.50×, lift 3.28× to 3.33×, 40.5–44.2 inspections per detected crash). A wider 25% flag rate yields PPV 1.37–1.48% (relative risk 3.00×, lift 1.99× to 2.00×, 67.5–73.1 inspections per crash), while empirical matched-set screening achieves PPV 0.96–1.04% (lift 1.40×, 96.4–104 inspections per

Table 9: From findings to design decisions in the AeroFleet 2.0 assurance tool, and how each part was checked.

| Finding | Design decision | Check |
|---|---|---|
| Logs did not attribute crashes (Q2); checks fired on the crash (Q4) | A flight review lists the log's events in time order and states the quality of its evidence; it names no cause | Unit and API tests |
| Roughness marked drones, not flights (Q3b, Q3c) | Drones ranked by vibration within a fleet, measured the same way; the top decile (10%) prioritized for inspection (or upper quartile, 25%) | Unit tests; no prospective evaluation yet |
| Useful evidence is longitudinal | Every upload, approval and review appended to a hash-chained, Ed25519-signed record with SHA-256 fingerprints of the files received | Tamper tests: edited, deleted, reordered and re-signed records detected |
| A plan proves nothing if something else was flown | Planned-versus-flown check: deviations beyond a fleet's corridor and height tolerance | Known deviations injected into 40 real flights (Appendix A.1) |
| Expert diagnoses as precedents (Q4) | Description search over 161 expert-labelled threads; checklist in order of expert frequency | Leave-one-out test on 96 cases (Appendix A.2) |
| Separation and deviations are three-dimensional | Tabletop 3D replay of a reviewed flight and rehearsal of planned missions, on screen or in a headset | Functional checks only; no user study (Appendix A.3) |

crash). The flag is therefore a maintenance inspection prompt, not an operational crash warning.

*A record that can be checked.* Because the useful evidence accumulates per aircraft, results are appended to a hash-chained record in which each entry is signed and carries the SHA-256 of the exact files it came from; each new head is also written to a separate file intended to be copied elsewhere. A standalone verifier checks an exported record; in tests it detected an edited field, a deleted entry, a reordering, and a chain rebuilt with a new key (the last only against a pinned key fingerprint or the separately kept heads, which is the stated limit of the design).

## 7. Discussion

*Attribution.* A declared rule, its held-out replication and cross-validated models all found no usable separation between pilot- and aircraft-caused crashes with these labels and features. Because pilot labels are sparse ($n = 36$ in the modeling sample), the permutation null distribution establishes a 95% detection threshold of 0.660; observed cross-validated models yield empirical 95% bootstrap upper bounds between 0.510 and 0.619, well below this threshold. Furthermore, independent evaluation against 43 expert forum cases revealed poor agreement on decided cases (balanced accuracy 0.381, 95% CI 0.20–0.59; pilot recall 1 of 5). While commercial and legal disputes often seek a binary verdict, aviation safety science demonstrates that accidents are multi-causal systemic events involving organizational, environmental and human factors [7]. Uploader labels are noisy, and the sharp divergence between uploader pilot blame (32%) and expert forum diagnoses (6%) points to politeness bias on public forums or a narrower definition of pilot fault among autopilot engineers. A log records the combined physical outcome of pilot inputs, environmental disturbances and vehicle response; disentangling responsibility from telemetry alone is not supported by the data. Tools that issue automatic responsibility verdicts should be evaluated against independent investigations before adoption.

*Leakage and confounding.* While cross-sectional models showed high apparent separation (AUC 0.88), this was likely influenced by airframe confounds and duration differences (duration alone yields AUC 0.662); within-drone

evaluation demonstrated substantial attenuation (AUC 0.580, 95% CI 0.528–0.633), confirming that longitudinal baseline comparison is essential. Diagnostic checks also exhibited leakage: specifically, tuning fault detection fell from 77.8% to 11.1% sensitivity ($p = 0.031$ on paired logs) when the final 10 s were removed. However, as noted in Section 5.6, this finding is exploratory and does not survive Bonferroni correction over five checks ($p \approx 0.16$), and the low baseline sensitivity of the remaining checks ($\leq 25\%$) leaves open whether other physical failure modes also leak. Diagnostic evaluations must remove the crash window and compare against an aircraft's own operating baseline before claiming pre-crash anticipation.

*Drone-level risk.* The positive result concerns airframes, not flights. Drones that later crashed vibrated more than comparable drones on ordinary flights, consistent with a maintenance view in which mechanical degradation precedes catastrophic failure [24]. However, two important qualifications apply. First, removing crash-day flights reduced usable sets from 60 to 25, indicating that many flights occurred during the same session; the "maintenance degradation" hypothesis therefore competes with an "experimental airframe" hypothesis, where poorly tuned or prototype drones vibrate heavily and crash during initial testing. Second, the effect was supported on the older estimator metric only; a declared replication on the newer accelerometer metric yielded an independent estimate of $\bar{r} = 0.50$ ($p = 0.5760$, 10 new sets) and pooled $\bar{r} = 0.58$ ($p = 0.251$, 19 sets), which is inconclusive and severely underpowered ($N = 247$ sets needed at true $\bar{r} = 0.58$ due to potential winner's curse). Finally, across the 0.689% screenable multi-flight base rate and 0.741% fleet vehicle base rate, stratum-matched fleet screening prioritizes the top decile with an operational PPV of 2.26–2.47% (lift 3.3×, 40.5–44.2 inspections per detected crash), and the top quartile with PPV 1.37–1.48% (lift 2.0×, 67.5–73.1 inspections per crash; 0.96–1.04% under empirical matched-set screening). The metric provides a maintenance prioritization signal, not an operational warning.

*From findings to a tool.* Section 6 turns these results into design decisions for AeroFleet 2.0, an operator's ground-side assurance tool. The common thread is that the tool shows evidence and ranks drones, and does not name a cause.

*Language models.* On this task small local models added errors and voting vulnerabilities rather than accuracy. While deterministic numeric parsing

prevented direct prompt injection by construction, multi-agent voting consensus was easily manipulated by a single compromised reviewer. Larger models and blind evaluation cases remain to be tested.

*Relation to prior datasets.* Benchmarks built from injected faults [8, 9] answer whether a known fault can be detected; anomaly datasets from real logs [10] answer whether unusual segments can be found. Our results concern a different question, attribution of a real crash, and suggest that performance on the first two does not transfer to it: signals that mark a fault also mark the crash it causes and the pilot's reaction to it. Datasets that pair real crashes with independently verified causes would be needed to make progress on attribution, and our expert-diagnosis labels are a small first step.

*Records and regulation.* Record-keeping rules ask operators to keep logs, maintenance records and occurrence reports [1, 2]. On our evidence, such records are most useful when they are kept per aircraft over time, so that an airframe's condition can be compared with its own history and with comparable airframes, rather than examined only after an accident.

*Practical guidance.* For operators and investigators we suggest: (i) treat automatic pilot-or-aircraft verdicts from logs with caution; (ii) use logs to assemble a time-ordered record of events for a human investigator; (iii) track vibration per drone across flights and inspect drones that are rough relative to comparable airframes; (iv) when evaluating any log-based diagnostic, remove the crash window and compare against the same drone's history.

*Future work.* Two studies have been declared in the repository before any data were collected: a prospective test in partner fleets, in which the monthly vibration ranking of AeroFleet 2.0 is written to the signed record before outcomes occur and crashes, unplanned repairs and defects found at inspection are compared between flagged and unflagged drones over six months; and a within-subjects user study comparing the three-dimensional views with a two-dimensional map on tasks with known answers. Stronger replication of the between-drone vibration effect on modern autopilots, evaluating operational safety impact across active fleets, testing sequence models or frequency-domain IMU features for transient precursors that whole-window summaries miss, and evaluating a larger set of crashes with causes verified by independent investigators would further strengthen the evidence.

## 8. Threats to validity

*Archive self-selection.* The public Flight Review archive reflects voluntary uploads from hobbyists, academic researchers, and commercial operators, introducing self-selection towards unusual flights, developer tuning sessions, and unexplained incidents. Crashed airframes and normal comparison flights differ substantially in flight duration (median 54 s vs. 388 s), firmware era, and operational envelope, which motivated our within-drone and matched-pair designs.

*Labels.* The uploader-assigned responsibility labels are not independently verified ground truth; they reflect the uploaders' own assessments, and normal flights are self-rated. Across 148 expert forum crash threads, download attempts yielded 43 downloadable logs (38 aircraft-fault, 5 pilot-fault), enabling a two-class empirical validation that revealed an overall error rate of 48.6% on decided cases (balanced accuracy 0.381, 95% bootstrap CI 0.20–0.59; pilot recall 1 of 5, Wilson 95% CI 3.7–71.0%; uploader `crash_pilot` rating 0 of 43). However, forum diagnoses represent informed community opinion rather than official accident investigation reports; original posters confirmed the diagnosis in only 47 of 94 aircraft threads (50.0%) and 5 of 9 pilot threads (55.6%). Controls without a crash rating may have crashed unnoticed, biasing the between-drone test conservatively. Forum diagnoses achieved good inter-rater agreement ($\kappa \geq 0.73$), but were restricted to public discussions.

*Confounding and design.* Matching on vehicle type, controller hardware and PX4 version leaves open differences in propellers, payload, flight style or operator identity, leaving potential unmodeled clustering. Results may not transfer to commercial ArduPilot fleets or to proprietary autopilots, even though the tool is designed to support both ArduPilot and PX4 logs.

*Measurement.* The crash moment is estimated; the within-drone result did not depend on the threshold. The between-drone effect was shown mainly with the older vibration estimate and was not confirmed on the newer metric in a declared replication with 19 usable sets; this main positive result needs stronger replication on newer firmware.

*Protocols and registration.* Protocols were declared and committed to version control in advance of analysis, and archived snapshots were deposited in Software Heritage (`swh:1:rev:bad06ff1256865c7e63d44389aac174e0281291d`,

swh:1:rev:bbdd0420de4547ac7d67b84165292a46237b604d, swh:1:rev:624dd694e0d207dd9c44b1c9550faaf04453bc56, and swh:1:rev:3a9aa89417fa9ea2a2334e3a4794e2358fb713ee); however, commit timestamps in self-hosted git remain author-controlled, distinguishing declared protocols from formal third-party timestamped registries (such as the Open Science Framework). One amendment (lifting the within-drone firmware filter) was data-dependent after observing sample starvation ($n = 12$ under the initial $\geq 1.10$ filter).

*Features.* Summary features over the whole window (percentiles, shares of time, RMS) were used; sequence models or frequency-domain IMU features might capture short precursors that summaries miss. This was not tested.

*Attrition.* Across 148 real crash threads, download attempts yielded 43 downloadable logs; exactly 37 of the 73 logs behind expert aircraft diagnoses with a checkable cause could still be downloaded, so several diagnostic comparisons in Table 7 have very small sample sizes ($n = 2$ to 9); its results are indicative rather than definitive.

*Power and sizing.* Some comparisons are small (14 pilot-caused matched sets, 12 sets with the newer vibration metric, five vibration-diagnosed logs). Null results there are not evidence of no effect. For the newer accelerometer vibration metric, the independent sample of 10 new cases yielded $\bar{r} = 0.50$ ($p = 0.5760$), and the pooled 19 sets yielded $\bar{r} = 0.58$ ($p = 0.251$). Statistical design power at $N = 19$ for an assumed effect of $\bar{r} = 0.67$ was 0.379 (and 0.118 if $\bar{r} = 0.58$). Detecting $\bar{r} = 0.67$ with 80% power requires 55 sets assuming the discovery effect was uninflated, but if winner's curse inflated the initial estimate and the true effect is $\bar{r} = 0.58$, 247 matched sets are required. Because 247 sets exceed the capacity of the public archive, definitive replication requires multi-fleet partner deployments.

*Language models.* The AI comparison is preliminary: the experiments use 60 author-generated cases and relatively small local models (3–4B parameters). The fixed rule's performance may be optimistic on cases created by the same authors who designed the rule (0 wrong answers is close to a training score); a blind set of cases written by someone who has not seen the rule is in preparation. Larger models or alternative prompt architectures were not tested.

*The tool.* AeroFleet 2.0's checks are primarily engineering tests on archive data and synthetic fault injections, not evidence of improved operational safety. Plans in the injection test were built from the flights themselves; the description search was tested on forum titles, which are shorter and more varied than an operator's report, and its symptom lists were written by the authors; prospective fleet evaluation of the vibration ranking and user studies of the three-dimensional views remain future work (Sections 6 and 7).

## 9. Conclusion

Across 590 crash-rated logs from an archive index of 479,547 flights, flight logs did not distinguish pilot-caused from aircraft-caused crashes above chance: cross-validated AUC upper bounds (0.510–0.619) remain below the 0.660 permutation null detection threshold, and two-class expert validation yielded poor agreement (balanced accuracy 0.381, 95% CI 0.20–0.59; pilot recall 1 of 5). Configuration protections were disabled at similar rates, and the flight that crashed showed no detectable difference in vibration or rate-tracking error relative to the same drone's prior flights (95% CI upper bounds 0.58 and 0.63). Diagnostic checks also suffered from crash-impact data leakage: tuning fault detection fell from 77.8% to 11.1% sensitivity ($p = 0.031$ on paired logs) once the final 10 s were removed. What the logs did reveal, on older estimator metrics, was that drones that later crashed exhibited higher baseline vibration on ordinary flights than matched peers, though replication on newer accelerometer metrics was inconclusive ($p = 0.25$). Screening against stratum-matched fleet controls achieves an operational positive predictive value of 2.26–2.47% (lift 3.3×, 40.5–44.2 inspections per crash) at the top decile and 1.37–1.48% (lift 2.0×) at a 25% flag rate across screenable (0.69%) and fleet-wide (0.74%) vehicle base rates (0.96–1.04% under empirical matched-set screening). Flight log analytics for drone fleets should prioritize longitudinal airframe screening and structured evidence review, not automated verdicts. AeroFleet 2.0 implements this approach as an open-source tool, presenting verifiable evidence while leaving diagnostic decisions to human operators.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work

reported in this paper. The authors develop the open-source AeroFleet 2.0 tool described in Section 6; it is not commercial.

## Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used an AI assistant (Claude, Anthropic) to write analysis code, draft and revise text, and assist the recording of forum diagnoses. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

## Data availability

Selection scripts, analysis code, the declared hypotheses with their commit history, derived features, expert labels (without forum text), and the source code of the AeroFleet 2.0 tool are available at `https://github.com/AdityaPathare46/aerofleet`; a DOI-archived release will be provided on acceptance. Raw logs are public on PX4 Flight Review and are referenced by identifier.

## Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

## Appendix A. Assurance tool implementation and validation checks

*Appendix A.1. Planned versus flown trajectory checks*

The check aligns the flown track with the plan's legs in flying order over the whole flight and reports stretches beyond the fleet's sideways and height tolerances (defaults 30 m and 15 m for at least 3 s). To test it, plans were built from 40 real multirotor flights in the archive (a waypoint every 10 s of track), so that the unmodified flight conformed by construction, and a known deviation was injected for 30 s. Sideways shifts of 35 m or more were found in 90–98% of flights and upward shifts of 20 m or more in 92–98%, while shifts below the tolerances were mostly not reported, as specified. The first version

of the check, which matched each position to the nearest leg anywhere, raised false alarms on 16 of 40 unmodified flights and detected only half of the 35 m shifts; it was replaced. The current version raised false alarms on 9 of 40 (22%, 95% CI 12–38%), all on fast flights cutting the corners of long legs built from the track; with a waypoint every 5 s (a post-hoc check) there were none. A declared check on the archive logs that record the mission they flew found only six distinct missions; the check reported no deviation while any of them flew in mission mode, and the two deviations it did report were stretches in which the vehicle had left mission mode (return-to-launch, or the pilot taking over). Six missions are too few for a rate (95% upper bound 39%), so the false-alarm rate on real plans remains unmeasured and a reported deviation is a reason to look at the replay rather than a finding.

*Appendix A.2. Searching past community diagnoses*

An operator can describe a problem in words. The description is matched (BM25, no model) against the expert-labelled forum threads, represented by their titles and up to 30 descriptive terms from the opening post, and against a short list of symptom words for each cause. After a first release we declared and ran a test, holding out each of the 96 threads in which the expert blamed the aircraft and named a cause and querying with the rest. The cause the words matched best was the expert's cause in 19 of 96 cases with the title alone (20%) and 21 of 96 with the title and opening-post terms (22%, 95% CI 15–31%), below the 26 of 96 (27%) of always answering the most frequent cause; among the first three it was 54% against 62%. The declared criterion (10 points above the base rate) was not met, as it was not with the first recording (20% against 32%). A written description, like the log, did not reveal the cause. The tool therefore orders its checklist by how often experts found each cause, marks the causes the description mentions, and offers the most similar threads only as reading, with the measured accuracy shown beside the answer.

*Appendix A.3. Three-dimensional replay and mission rehearsal*

Deviations, heights and separation between drones are spatial, and a two-dimensional plot hides height. The tool therefore draws flights on a virtual tabletop, a world in miniature [32], that can be viewed in a browser, in immersive virtual reality or, on a passthrough headset, in mixed reality on a real desk (WebXR).

A *flight replay* shows a reviewed flight: the approved plan dashed, the flown track green within tolerance and red during a deviation, the log's events pinned where they happened, and a playhead that can be scrubbed or played at ten times real speed.

A *mission rehearsal* shows planned missions before anyone flies: the deconfliction check turns each mission file into a position-over-time trajectory on a common clock (straight legs at stated climb, cruise and descent speeds), finds every pair closer than the separation distance and resolves conflicts by a launch delay or, where allowed, a different cruise height. The rehearsal flies all missions on that clock, as uploaded and as rescheduled, with each loss of separation drawn where and when it occurs (Figure A.7). It is a kinematic rehearsal of the plan, not a simulation of the aircraft: there is no wind, no vehicle dynamics and no sensor error, so it shows what the plan implies if it is flown as written, which is the assumption the planned-versus-flown check then tests after the flight.

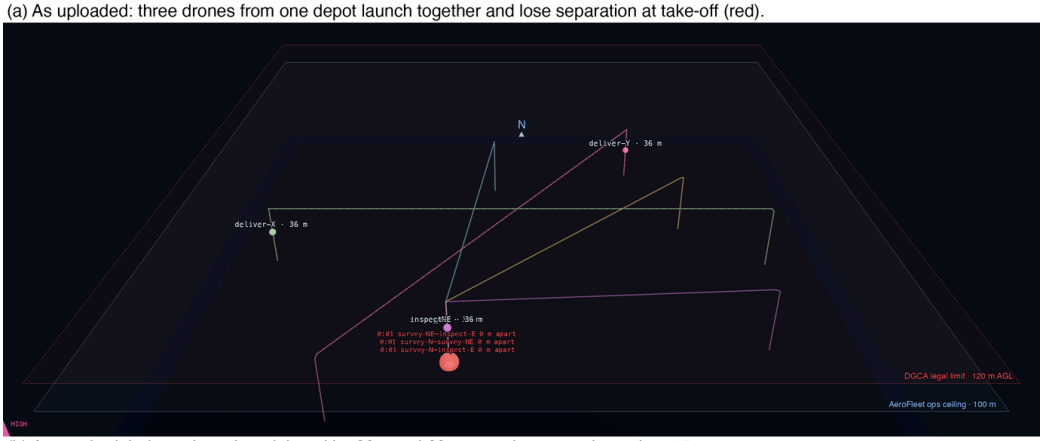
The replay and rehearsal have been checked functionally on desktop; a paired standalone headset application compiles but has not been run on a headset, and no user study has been carried out, so we make no claim that the three-dimensional view improves decisions.

## Appendix B. Exploratory evaluation of language-model review panels

Because language-model assistants are increasingly proposed for log review, we conducted a preliminary comparison on 60 author-generated fault cases using four relatively small local models (3–4B parameters). The fixed rule was right in 45 of 60 generated cases (75%, 95% Wilson interval 63–84%) and wrong in none (0%, 95% interval 0–6%); it referred the other 15 to a human operator. "Right" includes leaving the 10 no-cause cases open, so the rule named the true cause in 35 of the 50 cases that had one. As discussed in Section 8, the rule's performance may be optimistic because the cases were created by the authors who designed the rule; nonetheless, no model's panel matched it (Table B.10): wrong answers ranged from 6 to 38 of 60 (10–63%).

Reporting coverage (share of cases decided) and accuracy-when-answering separately clarifies the failure mechanisms of individual models. The single-agent configurations of `gemma3:4b` and `phi4-mini` abstained on all 50 cases with an active cause (0% active coverage); their 10 "right" scores stemmed

(a) As uploaded: three drones from one depot launch together and lose separation at take-off (red).




(b) As rescheduled: two launches delayed by 30 s and 60 s; no pair comes closer than 15 m.

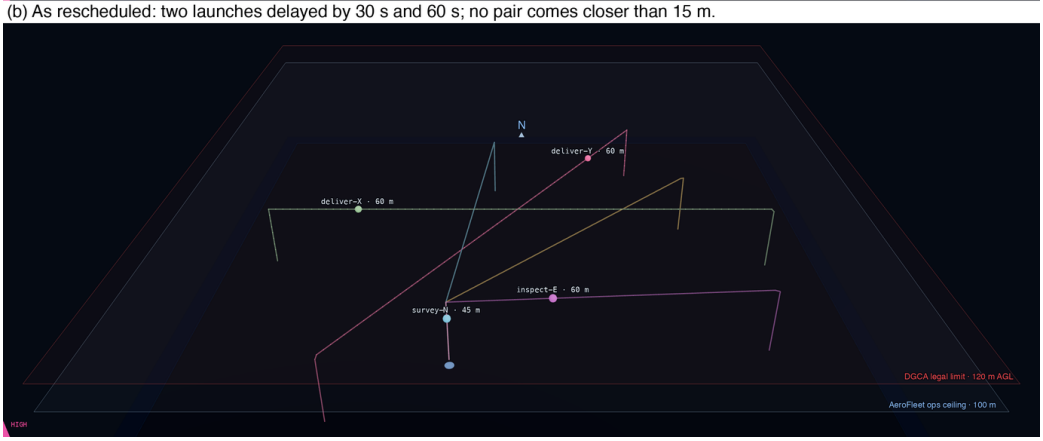



Figure A.7: Mission rehearsal in the tool's tabletop view (desktop rendering; the same scene opens in a WebXR headset). Five planned missions over a 2 km table, heights exaggerated four times, each mission in its own colour. (a) As uploaded, 12 s after launch: three missions leaving one depot together occupy the same point (overlapping labels) and the red sphere marks their loss of separation. (b) The schedule produced by the deconfliction check, 45 s after the first launch: two launches are delayed and no pair comes within 15 m. Straight legs at planned speeds, no wind.

entirely from correctly leaving the 10 no-cause cases open. `qwen2.5:3b` answered 53 of 60 cases (88% coverage) but achieved only 17.0% accuracy when answering (9 right, 44 wrong), which is indistinguishable from random guessing across the six cause classes (16.7%). `qwen3:4b` suffered complete mode collapse on decided cases: across the 41 cases it answered (the remaining 19 failed JSON schema parsing), it attributed all 41 to `PLAN_OR_ROUTE_ISSUE`, yielding 0 right, 41 wrong (0% accuracy when answering).

Multi-agent panel deliberation increased coverage for `gemma3:4b` and `phi4-mini` to 65%, but accuracy when answering remained low for `gemma3` (7.7%, 3 of 39) and modest for `phi4` (51.3%, 20 of 39). `qwen2.5:3b` achieved 62.5% accuracy when answering in the panel (10 right, 6 wrong), but deferred on 73% of cases (16 answered). `qwen3:4b` reached 30.9% accuracy when answering (17 of 55).

In the adversarial robustness test, we conducted a worst-case sensitivity sweep where a single compromised reviewer systematically advocated for a designated incorrect cause across both voting rounds. This adversary successfully flipped the panel's consensus in 45 to 56 of 60 cases (Table B.10). This highlights that multi-agent consensus voting without cryptographic proofs or fact-citation enforcement remains vulnerable to rogue reviewers. In contrast, 40 hostile prompt-injection strings planted in uploader-controlled fields never reached any model prompt because the pipeline deterministically extracts numeric telemetry into code, preventing direct prompt injection by construction.

Table B.10: Language-model review panels on 60 generated cases: counts of right (R), wrong (W), and deferred (Def) cases, with coverage (Cov., share of cases decided) and accuracy when answering (Acc.). "Right" includes correctly leaving a no-cause case open. Single-agent gemma3 and phi4 answered only the 10 no-cause cases (0% coverage on active causes). Adversary flips show consensus changes caused by a single compromised reviewer in a worst-case sweep.

| | Single agent | | | Review panel | | | Compromised |
|---|---|---|---|---|---|---|---|
| Method | R / W / Def | Cov. | Acc. | R / W / Def | Cov. | Acc. | Flips / 60 |
| Fixed rule (no model) | 45 / 0 / 15 | 75% | 100% | – | – | – | – |
| `gemma3:4b` | 10 / 0 / 50 | 17%† | 100%† | 3 / 36 / 21 | 65% | 7.7% | 49 |
| `phi4-mini` | 10 / 0 / 50 | 17%† | 100%† | 20 / 19 / 21 | 65% | 51.3% | 56 |
| `qwen2.5:3b` | 9 / 44 / 7 | 88% | 17.0% | 10 / 6 / 44 | 27% | 62.5% | 54 |
| `qwen3:4b` | 0 / 41 / 19 | 68% | 0.0% | 17 / 38 / 5 | 92% | 30.9% | 45 |

†Answered only on the 10 no-cause cases where abstention coincided with the ground truth; 0% coverage on active causes.

## References


[1] European Commission, Commission Implementing Regulation (EU) 2019/947 of 24 May 2019 on the rules and procedures for the operation of unmanned aircraft, Official Journal of the European Union, L 152, 11 June 2019 (2019).

[2] European Union Aviation Safety Agency, Easy Access Rules for Unmanned Aircraft Systems (Regulations (EU) 2019/947 and 2019/945), EASA, Cologne, revision of June 2026 (2026).

[3] Ministry of Civil Aviation, Government of India, The Drone Rules, 2021, G.S.R. 589(E), Gazette of India, 25 August 2021 (2021).

[4] Federal Aviation Administration, 14 CFR Part 107: Small Unmanned Aircraft Systems, Electronic Code of Federal Regulations, as of 21 October 2026 (2026).

[5] V. S. A. Duvvuru, B. Zhang, M. Vierhauser, A. Agrawal, LLM-agents driven automated simulation testing and analysis of small uncrewed aerial systems, in: 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), 2025, pp. 385–397. doi:10.1109/ICSE55347.2025.00223.

[6] S. Silalahi, T. Ahmad, H. Studiawan, F. Breitinger, DroPTC: Sentence-level drone flight log forensics using contrastive learning and explainable AI, Forensic Science International: Digital Investigation 56 (2026) 302051. doi:10.1016/j.fsidi.2026.302051.

[7] D. A. Wiegmann, S. A. Shappell, A Human Error Approach to Aviation Accident Analysis: The Human Factors Analysis and Classification System, Ashgate, Aldershot, 2003.

[8] A. Keipour, M. Mousaei, S. Scherer, ALFA: A dataset for UAV fault and anomaly detection, The International Journal of Robotics Research 40 (2–3) (2021) 515–520. doi:10.1177/0278364920966642.

[9] X. Le, B. Jin, G. Cui, X. Dai, Q. Quan, RflyMAD: A dataset for multicopter fault detection and health assessment, The International Journal of Robotics Research 44 (7) (2025) 1081–1092. doi:10.1177/02783649241305153.

[10] A. Kabaoglu, S. Sariel, UAV-SEAD: State estimation anomaly dataset for UAVs, arXiv preprint https://arxiv.org/abs/2602.13900 (2026).

[11] G. Wild, J. Murray, G. Baxter, Exploring civil drone accidents and incidents to help prevent potential air disasters, Aerospace 3 (3) (2016) 22. doi:10.3390/aerospace3030022.

[12] Air Accidents Investigation Branch, Annual Safety Review 2025, Tech. rep., AAIB, Department for Transport, Farnborough, UK, published 4 June 2026 (2026).

[13] S. Kaufman, S. Rosset, C. Perlich, O. Stitelman, Leakage in data mining: Formulation, detection, and avoidance, ACM Transactions on Knowledge Discovery from Data 6 (4) (2012) 15:1–15:21. doi:10.1145/2382577.2382579.

[14] S. Kapoor, A. Narayanan, Leakage and the reproducibility crisis in machine-learning-based science, Patterns 4 (9) (2023) 100804. doi:10.1016/j.patter.2023.100804.

[15] L. Meier, D. Honegger, M. Pollefeys, PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms, in: 2015 IEEE International Conference on Robotics and Automation (ICRA), 2015, pp. 6235–6240. doi:10.1109/ICRA.2015.7140074.

[16] PX4 Development Team, Flight Review log archive and pyulog ULog parser, https://review.px4.io; pyulog v1.2.4, https://github.com/PX4/pyulog; BSD-3-Clause (2026).

[17] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, D. T. Mellor, The preregistration revolution, Proceedings of the National Academy of Sciences 115 (11) (2018) 2600–2606. doi:10.1073/pnas.1708274114.

[18] J. Wei, H. Li, Y. Li, Y. Zhao, D. Yao, B. Lu, X. Ye, S.-K. Im, Y. Wang, X. Yang, AeroTSBoost: Temporal-statistical boosting for real-world UAV telemetry anomaly mining, arXiv preprint https://arxiv.org/abs/2605.25639 (2026).

[19] K. Kose, J. Wing, N. A. Kose, C. Guadarrama-Trejo, A. Sowers, A. Rasheed, A UAV testbed for diagnosing hardware vulnerabilities:

Quantifying sim-to-real discrepancies in PX4 flight logs, Sensors 26 (10) (2026) 3188. doi:10.3390/s26103188.

[20] R. Puchalski, W. Giernacki, UAV fault detection methods, state-of-the-art, Drones 6 (11) (2022) 330. doi:10.3390/drones6110330.

[21] E. Khalastchi, M. Kalech, On fault detection and diagnosis in robotic systems, ACM Computing Surveys 51 (1) (2018) 9:1–9:24. doi:10.1145/3146389.

[22] E. Zibaei, R. Borth, Building causal models for finding actual causes of unmanned aerial vehicle failures, Frontiers in Robotics and AI 11 (2024) 1123762. doi:10.3389/frobt.2024.1123762.

[23] D. Wang, S. Li, G. Xiao, Y. Liu, Y. Sui, An exploratory study of autopilot software bugs in unmanned aerial vehicles, in: Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), 2021, pp. 20–31. doi:10.1145/3468264.3468559.

[24] E. Petritoli, F. Leccese, L. Ciani, Reliability and maintenance analysis of unmanned aerial vehicles, Sensors 18 (9) (2018) 3171. doi:10.3390/s18093171.

[25] A. P. Tvaryanas, W. T. Thompson, S. H. Constable, Human factors in remotely piloted aircraft operations: HFACS analysis of 221 mishaps over 10 years, Aviation, Space, and Environmental Medicine 77 (7) (2006) 724–732.

[26] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, I. Mordatch, Improving factuality and reasoning in language models through multiagent debate, in: Proceedings of the 41st International Conference on Machine Learning (ICML), Vol. 235 of PMLR, 2024, pp. 11733–11763.
URL https://proceedings.mlr.press/v235/du24e.html

[27] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, M. Fritz, Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection, in: Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec), 2023, pp. 79–90. doi:10.1145/3605764.3623985.

[28] B. Frénay, M. Verleysen, Classification in the presence of label noise: A survey, IEEE Transactions on Neural Networks and Learning Systems 25 (5) (2014) 845–869. doi:10.1109/TNNLS.2013.2292894.

[29] E. A. Stuart, Matching methods for causal inference: A review and a look forward, Statistical Science 25 (1) (2010) 1–21. doi:10.1214/09-STS313.

[30] A. Aamodt, E. Plaza, Case-based reasoning: Foundational issues, methodological variations, and system approaches, AI Communications 7 (1) (1994) 39–59. doi:10.3233/AIC-1994-7104.

[31] S. Robertson, H. Zaragoza, The probabilistic relevance framework: BM25 and beyond, Foundations and Trends in Information Retrieval 3 (4) (2009) 333–389. doi:10.1561/1500000019.

[32] R. Stoakley, M. J. Conway, R. Pausch, Virtual reality on a WIM: Interactive worlds in miniature, in: Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '95), 1995, pp. 265–272. doi:10.1145/223904.223938.

[33] J. A. Hanley, B. J. McNeil, The meaning and use of the area under a receiver operating characteristic (ROC) curve, Radiology 143 (1) (1982) 29–36. doi:10.1148/radiology.143.1.7063747.

[34] H. B. Mann, D. R. Whitney, On a test of whether one of two random variables is stochastically larger than the other, The Annals of Mathematical Statistics 18 (1) (1947) 50–60. doi:10.1214/aoms/1177730491.

[35] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, New York, 1993.

[36] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.
URL https://www.jstor.org/stable/4615733